

Week 01 — Lab: Be the Web

100L • Semester 1 • Lab time: 100 min

Deliverable (curriculum §4, week 1): a diagram showing User → Browser → Request → Server → Database → Response → Browser.

You will build up to the diagram in three steps:

Part	Time	Format	What you get out of it
1. Be the Web	35 min	Whole class, on your feet	You <i>act out</i> a page load, so you have done every step yourself
2. What broke?	20 min	Groups of 4	You work backwards from a symptom to the step that failed
3. The diagram	45 min	Individual	The deliverable

No laptop, phone or internet connection is needed for any part.

Part 1: Be the Web (35 min)

The class becomes the Internet. Every student gets a role card. The tutor narrates, and messages travel on paper.

Roles (for about 12 students; run several groups in parallel if the class is bigger)

Role	How many	What you do
User	1	Wants to see their results. Says "I typed <code>myportal.ng/results</code> ."
Browser	1	Does all the asking on the User's behalf. The only role that talks to the User.
Resolver	1	Answers "where is ...?" by asking the three DNS roles below
Root server	1	Only knows who handles each domain ending ("ask the .ng server")
.ng server	1	Only knows who handles each <code>.ng</code> name ("ask myportal.ng's DNS")
myportal.ng DNS	1	Knows the address: <code>203.0.113.25</code>
Web server	1	Sits at seat <code>203.0.113.25</code> . Holds the certificate card. Answers requests.
Database	1	Holds the results sheet. Only talks to the Web server.
Network couriers	3–4	Carry every slip between Browser and Web server. They may not skip anyone.

Everyone else **watches and keeps score**: they count every slip that crosses the room.

Round 1: plain HTTP (10 min)

- User → Browser: "Open `myportal.ng/results`."
- Browser → Resolver: the **DNS question slip**. The Resolver walks to Root, then the `.ng` server, then myportal.ng DNS, and returns with `203.0.113.25`.

3. Browser writes the **request slip** (`GET /results` , `Host: myportal.ng`) and hands it to a courier.
4. The courier carries it, *and is allowed to read it aloud*. This is HTTP.
5. Web server → Database: "Results for matric no. 2026/0001?" The Database answers with the rows.
6. Web server writes the **response slip**: `200 OK` and an HTML page. The tutor **tears it into three numbered strips** (packets) and gives each to a different courier.
7. The tutor quietly tells one courier to "lose" their strip. The Browser must notice the gap and ask for strip 2 again.
8. The Browser reassembles the strips in order and reads the page to the User.

Round 2: HTTPS (10 min)

Repeat Round 1 with two changes:

- **Slips go in sealed envelopes.** Couriers can read only the address on the outside. Ask them: "What do you know about this message now?"
- **Before any envelope moves**, the Web server shows its **certificate card**: "I am `myportal.ng` , signed by Trusted Authority." The Browser checks that the name matches.

The impostor: one courier secretly holds a fake certificate that reads "I am `myportal.ng` ." The courier tries to answer the Browser in place of the real server. The Browser must refuse.

Round 3: one page, many requests (5 min)

This time the response HTML contains `<link href="style.css">` and `` . The Browser must send **two more requests** before the page is complete. The scorekeepers announce the final count of slips for one page view.

Debrief (10 min): answer in your notebook

1. Which roles never spoke directly to the User? Why does that matter for security?
2. In Round 1, what could a courier have done with the slip besides read it?
3. Which step would have happened only once if the Browser had remembered the address from earlier? (That memory is called a **cache**.)
4. Which role played *client* in one exchange and *server* in another?
5. Total slips for one page view: _____. What would make that number grow on a real site?

Part 2: What broke? (20 min)

Each group of four gets nine scenario cards. For each card, decide **which step of the journey failed**:

`DNS` · `Connection` · `HTTPS/certificate` · `The request (4xx)` · `The server (5xx)` · `Database` · `Follow-up files` · `The network in between` · `Cache`

Write a one-line reason for each answer.

#	Scenario
1	The browser says: "This site can't be reached. The server's DNS address could not be found."
2	The browser shows a full-page warning: "Your connection is not private."
3	You typed the portal address with a typo in the path. The page says 404 Not Found .
4	You click "View results" and get a plain page reading 500 Internal Server Error .
5	You log in fine, but the results area says "Could not connect to database."
6	The page loads but looks broken: plain black text, no colours, no layout, empty boxes where images should be.
7	The site works on your mobile data but not on the hostel Wi-Fi. Your friend's phone on the same Wi-Fi fails too.
8	The school updated the portal an hour ago. Your friend sees the new version, but you still see the old one.
9	The address is found, but the browser waits a long time then says "ERR_CONNECTION_TIMED_OUT."

The tutor reveals the answers ([key in the tutor notes](#)). Scoring is one point per card, and your reason has to be right too. The group with the highest score presents first in show & tell.

Part 3: The diagram (45 min): the deliverable

Work alone. Draw the journey you acted out in Part 1, for a real site of your choice.

It must have

- **These parts, in this order:** User → Browser → Request → Server → Database → Response → Browser.
- **DNS**, placed where it really happens, which is before the request.
- **Labelled arrows.** Every arrow says *what travels along it*: a question, an address, a request, a query, rows, an HTML page.
- **Asking and answering that look different.** For example, use solid arrows for requests and dashed arrows for responses, or two colours. Add a key.
- **HTTPS shown in the right place**, on the link between browser and server.
- **No more than 10 boxes.**

Underneath the diagram, write

1. **The journey story.** Five to eight sentences telling the journey in your own words, as if explaining it to a younger sibling.
2. **One question you still have.** It must be an honest one. "None" is not accepted.

Then: compare

Put your **Before sketch** (from the pre-read) next to your new diagram. In one sentence, write the biggest thing you got wrong or left out.

Optional extras (no extra marks)

- Add the follow-up requests from Round 3.
- Mark with ✗ where scenarios 1, 2 and 5 from Part 2 would break your diagram.
- Redraw it digitally at mermaid.live, which works in a phone browser with no account.

Submission

Git is taught in week 11. Until then, submit work by photo:

1. Photograph your diagram page (with the story and question), your Part 1 debrief, and your Before sketch.
2. Name the files `week-01-<surname>-<firstname>-<n>.jpg`, numbered 1, 2, 3.
3. Upload them to the class submission folder the tutor shares in the room, **before the next session**.

Keep the paper. You will add to this diagram again in later weeks, as the programme adds APIs, React and your own backend.

Marking (10 points)

Criterion	Points	What earns full marks
Complete, ordered journey	3	Every required part is present and in the right order
DNS and database placed correctly	2	DNS comes before the request; the database answers the server, not the browser
Arrows mean something	2	Every arrow is labelled; asking and answering are visually distinct, with a key
HTTPS placed correctly	1	On the browser-server link, not as a box of its own
Journey story	1	Matches the diagram, in the student's own words
Honest question + Before/After sentence	1	Both present

Part 1 and Part 2 are not graded. The debrief answers count only as evidence that you attended.

Tutor notes

Kit

Print one set per group of about 12:

- **Role cards**, a set per group, with the role name in large letters. Each card carries its one-line instruction from the roles table.
- **DNS question slip**: "Where is `myportal.ng`?"

- **Request slips** (×3): `GET /results` · `GET /style.css` · `GET /logo.png` , each with `Host: myportal.ng` .
- **Database slips**: the question "Results for matric no. 2026/0001?" and an answer slip with three made-up course rows.
- **Response slip**: `200 OK` · `content-type: text/html` , plus a short HTML page that includes a `<link>` and an `<img>` . Draw lines on it for tearing into three numbered strips.
- **Certificate cards**: one genuine card ("I am myportal.ng — signed: Trusted Authority") and one fake ("I am myp0rtal.ng").
- **Envelopes** for Round 2.

`myportal.ng` is a made-up name used only in the game, and `203.0.113.25` comes from a range reserved for examples.

Running the role-play well

- Make the couriers *walk*. Physical distance shows why a cache matters better than any explanation does.
- In Round 1, encourage a courier to "helpfully" edit the request slip. Then ask the room what an attacker on public Wi-Fi could do with that.
- If the Resolver forgets to visit the root first, let it happen, then ask "How did you know where the `.ng` server was?"
- **Large classes**: run several groups of 12 at once, each with its own kit. Observers rotate into roles between rounds.

Answer key (Part 2)

#	Failed step	Why
1	DNS	The name never turned into an address, so nothing else could happen
2	HTTPS/certificate	The connection was made, but the server's ID check failed. It may be an impostor or an expired certificate.
3	The request (4xx)	The server is fine and answered correctly; the client asked for a path that does not exist
4	The server (5xx)	The request was valid, but the server's own code failed
5	Database	The server is running, but its call to the database failed. The "Server → Database" arrow is the broken one.
6	Follow-up files	The HTML arrived, but the follow-up requests for CSS and images failed (Round 3)
7	The network in between	The same server works over another route, so this network is blocking the site or its resolver is failing
8	Cache	Your device (or your resolver) still holds an old copy, and the next fresh request will fix it
9	Connection	DNS gave an address, but nothing answered at it. The host may be down, or the server is not reachable.

What to look for in the diagrams

What you see	What it means	Prompt
The browser talks straight to the database	The backend's role as gatekeeper is missing	"Who decides whether you're allowed to see those rows?"
No DNS, or DNS after the request	Names and addresses are still blurred together	"Where did the Web server's seat number come from in Part 1?"
Every arrow points the same way	There is no response in the model	"How did the strips get back to the Browser?"
Google drawn between browser and site	Thinks the search engine sits on the path	"In Part 1, was there a Google role?"